LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Topic: Student Project Portfolio & Showcase App

NAME:PRATIK PATIL

SRN: PES2UG24CS372

SECTION: G

1. Objective

To evaluate suitable architectural styles for the Student Project Portfolio & Showcase App, select the most appropriate architecture, and model the system using a UML Component Diagram showing components, interfaces, dependencies, and interactions.

2. Scenario Overview

The Student Project Portfolio & Showcase App allows students to maintain and showcase their academic and personal projects. The application provides a web/mobile interface, authentication, student profile management, project management, and persistent storage.

3. Architectural Styles Considered

Layered Architecture provides clear separation of concerns but can introduce coupling between layers and makes independent scaling more difficult. Microservices Architecture divides the application into small, independently deployable services and supports independent scaling, fault isolation, and independent deployment, but introduces operational complexity and network latency. Client-Server Architecture provides centralized control and simple deployment but can create a single point of failure and a scalability bottleneck.

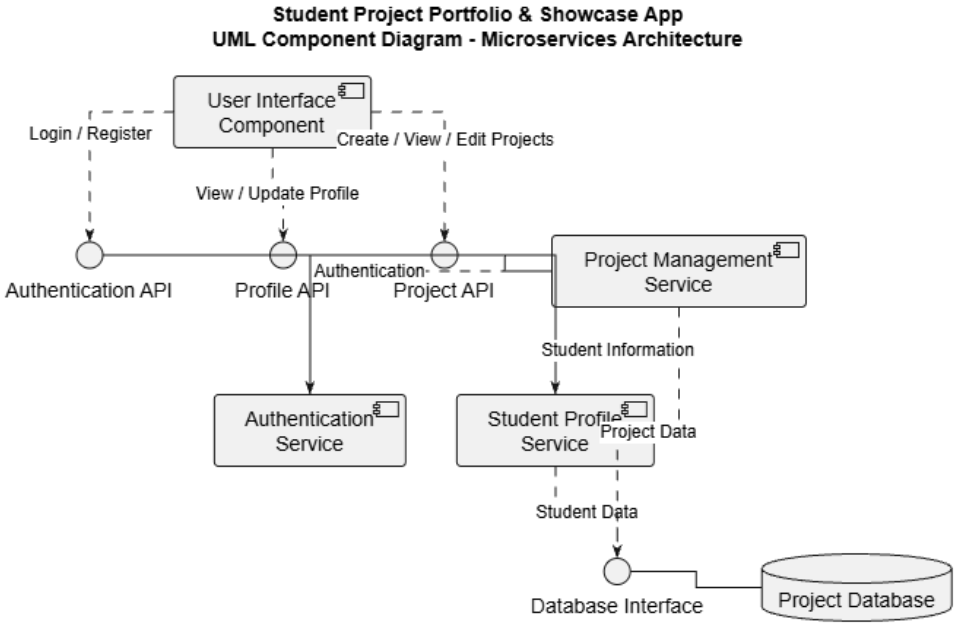
4. Selected Architecture

Microservices Architecture is selected for the Student Project Portfolio & Showcase App. The application is divided into separate services responsible for authentication, student profiles, project management, and data storage. These services communicate through well-defined interfaces.

5. Components Identified

1. User Interface Component – provides the web/mobile interface for students and visitors.
2. Authentication Service – handles registration, login, and authentication.
3. Student Profile Service – manages student profiles, portfolio information, and skills.
4. Project Management Service – creates, edits, deletes, and showcases student projects.
5. Project Database – stores student, project, and portfolio information.

6. Components Diagram



7. Interfaces and Interactions

Authentication API connects the User Interface Component with the Authentication Service for login and registration. Profile API connects the User Interface Component with the Student Profile Service for viewing and updating profiles. Project API connects the User Interface Component with the Project Management Service for creating, viewing, and editing projects. Database Interface connects application services with the Project Database for storing and retrieving information.

8. Architectural Justification

Independent development and deployment allow authentication, profile management, and project management services to be developed, tested, maintained, and deployed separately. Scalability and flexibility are improved because services can be scaled independently; for example, project showcase functionality can receive more traffic than profile editing.

9. Security Advantage

The Authentication Service isolates authentication and authorization responsibilities from other services. This helps protect user accounts and ensures that only authenticated users can perform protected operations such as creating or managing projects.

10. Performance Benefit

Frequently accessed functionality, particularly project viewing and showcasing, can be handled by scaling the relevant service independently. This improves resource utilization and helps maintain responsive performance as the number of users increases.

11. Conclusion

Microservices Architecture is an appropriate choice because it provides modularity, independent deployment, scalability, and flexibility. The architecture supports the identified components and interfaces while allowing the application to grow as additional portfolio and showcase features are introduced.